

Análisis de complejidad función por función

Proyecto 9 — Asignación de frecuencias (FAP) mediante coloreo de grafos
Caso de estudio: mapa del Perú (25 departamentos)

Grupo 10 — Análisis y Diseño de Algoritmos, UNMSM

Semestre 2026-I

Resumen

Se analiza, *función por función*, la complejidad temporal y espacial del código del Proyecto 9 (paquete `src/`). El método es uniforme: cada función se **divide en bloques**, a cada línea se le asigna un costo unitario c_i y un número de ejecuciones t_i , el tiempo se plantea como la **sumatoria** $T = \sum_i c_i t_i$ y esta se resuelve hasta una forma cerrada que se acota asintóticamente. Se usan además: identidades combinatorias (lema del apretón de manos), análisis amortizado (orden smallest-last, muestreo de $G(n, p)$), conteo agregado con borrado perezoso (DSATUR), árboles de recursión (backtracking) y esperanza matemática (generadores aleatorios). Se cubre el coloreo por distancia (recorrido BFS desde el departamento elegido) usado por la aplicación visual. El mapa, la adyacencia y los centroides se derivan de un GeoJSON oficial de los 25 departamentos.

Índice

1. Metodología	2
1.1. Modelo de cómputo	2
1.2. Método de bloques y sumatorias	2
2. core.py: grafo y verificación	2
2.1. Operaciones de costo constante o lineal directo	2
2.2. <code>edges()</code> : enumerar cada arista una vez	3
2.3. <code>is_proper(graph, color)</code>	3
3. algorithms.py: ordenamientos	3
3.1. <code>order_natural</code> : $\Theta(n)$	3
3.2. <code>order_largest_first</code> : $\Theta(n + \Delta)$	3
3.3. <code>order_smallest_last</code> : $\Theta(n + m)$ amortizado	4
3.4. <code>order_distance</code> y <code>bfs_levels</code> : coloreo por distancia	4
4. greedy	5
5. dsatur	6
6. backtracking: $\chi(G)$ exacto	6
6.1. <code>greedy_clique</code> : cota inferior polinomial	6
6.2. Podas (preservan completitud)	7
6.3. <code>_search</code> : recurrencia del árbol	7
6.4. <code>chromatic_number</code> : iterar sobre k	7
7. generators.py: instancias aleatorias	8
7.1. <code>gnp</code> : $G(n, p)$ en $\Theta(n + m)$ esperado	8
7.2. <code>geometric</code> : rejilla espacial	8

8. <code>peru.py</code> : el mapa	8
9. Tabla resumen	9
10. Contraste experimental	9
11. El caso $n = 10^{10}$: por qué es solo teórico	10

1. Metodología

1.1. Modelo de cómputo

Se usa el modelo **RAM** con criterio de costo uniforme: cada operación elemental cuesta una constante $c_i > 0$. El tamaño de un grafo $G = (V, E)$ se describe con $n = |V|$ y $m = |E|$; $\Delta(G)$ es el grado máximo y $\deg(v)$ el grado de v .

Observación 1.1 (diccionarios y conjuntos). El grafo se representa con **dict** y **set** (tablas hash): inserción, búsqueda y borrado cuestan $\mathcal{O}(1)$ *en promedio*. Como las claves son enteros que generamos nosotros (no adversariales), usamos ese costo promedio en todo el documento; las cotas son, en rigor, esperadas.

1.2. Método de bloques y sumatorias

Paso 1. Dividir en bloques $B_1, B_2, \dots$ (inicialización, bucle externo, bucle interno, retorno): un bloque es un fragmento cuyo número de repeticiones se describe con una sola expresión.

Paso 2. Costo por línea: costo unitario c_i y repeticiones t_i . Una cabecera de bucle que itera k veces aporta $t_i = k + 1$; su cuerpo $t_i = k$.

Paso 3. Plantear la sumatoria $T(n, m) = \sum_i c_i t_i(n, m)$. Si el costo de una iteración depende del vértice (p.ej. $\deg(v)$), la sumatoria se escribe sobre los vértices.

Paso 4. Resolver y acotar hasta forma cerrada y concluir $T = \Theta(f)$ (o $\mathcal{O}$ cuando solo se acota por arriba).

La identidad clave es el *lema del apretón de manos*:

$$\sum_{v \in V} \deg(v) = 2m, \quad (1)$$

pues cada arista aporta 1 al grado de sus dos extremos. Por ello “recorrer la vecindad de cada vértice una vez” cuesta $\Theta(n + 2m) = \Theta(n + m)$: la firma de los algoritmos lineales sobre grafos dispersos. Para funciones recursivas el Paso 3 da una *recurrencia* (se resuelve con árbol de recursión); para costos irregulares, *análisis amortizado*; para los generadores, costo *esperado*.

Definición 1.2. $f = \mathcal{O}(g)$ si $\exists c, n_0 : f(n) \leq c g(n) \forall n \geq n_0$; $f = \Omega(g)$ si $g = \mathcal{O}(f)$; $f = \Theta(g)$ si ambas.

2. core.py: grafo y verificación

La estructura es $\text{adj} : V \rightarrow 2^V$ con $\text{adj}[u] = N(u)$. **Espacio** $\Theta(n + m)$: n entradas más $\sum_v \deg(v) = 2m$ referencias (óptimo para grafos dispersos; una matriz costaría $\Theta(n^2)$, inviable para $n = 10^6$).

2.1. Operaciones de costo constante o lineal directo

Función	Costo	Justificación
<code>add_vertex</code>	$\mathcal{O}(1)$	<code>setdefault</code> en <code>dict</code> (obs. 1.1).
<code>add_edge</code>	$\mathcal{O}(1)$	dos <code>add_vertex</code> + dos inserciones en <code>set</code> .
<code>neighbors, degree</code>	$\mathcal{O}(1)$	referencia al conjunto / <code>len</code> .
<code>vertices, n</code>	$\mathcal{O}(1)$	vista / <code>len</code> del <code>dict</code> .
<code>m, max_degree</code>	$\Theta(n)$	suma/máximo de n longitudes almacenadas.
<code>from_edges</code>	$\Theta(n + m)$	n veces <code>add_vertex</code> + m veces <code>add_edge</code> .
<code>num_colors</code>	$\Theta(n)$	<code>set</code> de los n colores.

Para m : $T(n) = \sum_{v \in V} c_1 = \Theta(n)$ (longitudes ya almacenadas; no se recorren aristas).

2.2. edges(): enumerar cada arista una vez

```

1 def edges(self):
2     for u in self.adj:           # B1: n iteraciones
3         for v in self.adj[u]:    # B2: deg(u) iteraciones
4             if u < v:            # B2
5                 yield u, v       # B2 (solo m veces en total)

```

Bloque	Línea	Costo	Veces
B_1	2	c_2	$n + 1$
B_2	3	c_3	$\sum_u (\deg(u) + 1)$
B_2	4	c_4	$\sum_u \deg(u)$
B_2	5	c_5	m

Con (1):

$$T(n, m) = c_2(n+1) + c_3(2m+n) + c_4 2m + c_5 m = \Theta(n + m).$$

Cada arista se *visita* dos veces pero se *emite* una (cuando $u < v$): espacio extra $\mathcal{O}(1)$, sin el conjunto auxiliar de “vistas”.

2.3. is_proper(graph, color)

```

1 def is_proper(graph, color):
2     for u, v in graph.edges():    # B1: m aristas
3         if not color.get(u) or not color.get(v) \
4             or color[u] == color[v]:    # B1: O(1)
5             return False, (u, v)
6     return True, None              # B2

```

El generador entrega las m aristas en $\Theta(n + m)$ y cada una se procesa en $\mathcal{O}(1)$:

$$T(n, m) = \underbrace{\Theta(n + m)}_{\text{generador}} + \sum_{e \in E} \mathcal{O}(1) = \Theta(n + m) \quad (\text{peor caso: coloreo válido}).$$

Mejor caso $\Omega(1)$: sale en el primer conflicto. Aceptar exige mirar toda la entrada; rechazar puede ser inmediato.

3. algorithms.py: ordenamientos

Greedy separa (a) construir un orden de vértices y (b) colorear; $T_{\text{greedy}} = T_{\text{orden}} + T_{\text{coloreo}}$. Analizamos los cuatro órdenes.

3.1. order_natural: $\Theta(n)$

`list(g.vertices())` copia n referencias.

3.2. order_largest_first: $\Theta(n + \Delta)$

```

1 def order_largest_first(g):
2     buckets = [[] for _ in range(g.max_degree() + 1)]           # B1
3     for v in g.vertices():                                       # B2: n
4         buckets[g.degree(v)].append(v)                          # B2
5     return [v for d in range(len(buckets)-1, -1, -1)
6             for v in buckets[d]]                                # B3

```

B_1 crea $\Delta + 1$ cubetas ($+\Theta(n)$ de `max_degree`); B_2 mete cada vértice en su cubeta en $\mathcal{O}(1)$; B_3 recorre $\Delta + 1$ cubetas y mueve n elementos. Total

$$T(n) = \Theta(n) + c(\Delta+1) + c'n + \left(c''(\Delta+1) + \sum_d |B_d|\right) = \Theta(n + \Delta) \subseteq \Theta(n).$$

Punto fino: ordenar por grado con un sort comparativo costaría $\Theta(n \log n)$; como el grado es un entero en $[0, \Delta]$, el *bucket sort* lo hace lineal, dejando Greedy en $\Theta(n + m)$ estricto. (Welsh–Powell es el mismo orden.)

3.3. order_smallest_last: $\Theta(n + m)$ amortizado

```

1 def order_smallest_last(g):
2     deg = {v: g.degree(v) for v in g.vertices()}                # B1: n
3     buckets = [set() for _ in range(g.max_degree()+1)]          # B1
4     for v, d in deg.items(): buckets[d].add(v)                  # B1: n
5     removed, seq, i = set(), [], 0
6     for _ in range(g.n):                                         # B2: n iter.
7         while not buckets[i]: i += 1                             # B3: sube puntero
8         v = buckets[i].pop(); removed.add(v); seq.append(v)      # B4: O(1)
9         for u in g.neighbors(v):                                 # B5: deg(v)
10            if u not in removed:
11                buckets[deg[u]].discard(u); deg[u] -= 1
12                buckets[deg[u]].add(u)                            # B5: O(1)
13            i = max(i - 1, 0)                                       # B6: baja <= 1
14    seq.reverse(); return seq                                     # B7: n

```

B_1, B_4, B_7 suman $\Theta(n)$; B_5 suma $\sum_v \deg(v) = 2m$ iteraciones $\mathcal{O}(1)$, es decir $\Theta(m)$. El único costo no evidente es B_3 :

Lema 3.1 (costo amortizado del puntero). *El total de incrementos de i es $\leq n + \Delta$.*

Demostración. Sean U, D los totales de subidas y bajadas. El puntero parte en 0, termina en $i_f \leq \Delta$ y nunca es negativo, luego $U - D = i_f \leq \Delta$. Cada una de las n vueltas de B_2 baja a lo más una vez (B_6), así $D \leq n$ y $U \leq n + \Delta$. $\square$

Sumando: $T(n, m) = \Theta(n) + \mathcal{O}(n + \Delta) + \Theta(m) = \Theta(n + m)$.

Teorema 3.2 (Matula–Beck, 1983). *Greedy en orden smallest-last usa $\leq d(G) + 1$ colores, con $d(G)$ la degenerescencia. En grafos planos $d(G) \leq 5$, luego ≤ 6 colores.*

3.4. order_distance y bfs_levels: coloreo por distancia

La aplicación permite elegir un departamento *base s* y colorear “desarrollando” el color desde él: los departamentos se ordenan por distancia (número de fronteras) a s . Esa distancia se calcula con un recorrido en anchura (BFS); sobre un mapa plano la distancia en aristas sigue de cerca a la lejanía geográfica.

```

1 def order_distance(g, source):
2     seen = {source}; queue = deque([source]); seq = []
3     while queue:                                                 # B1: cada vertice sale 1 vez

```

```

4     v = queue.popleft(); seq.append(v)
5     for u in sorted(g.neighbors(v),
6                     key=g.degree, reverse=True): # B2: deg(v) + orden local
7         if u not in seen:
8             seen.add(u); queue.append(u)          # B2: O(1)
9     seq.extend(v for v in g.vertices() if v not in seen) # B3: n
10    return seq

```

Sumatoria. Cada vértice entra y sale de la cola exactamente una vez (B_1 : n iteraciones). En B_2 se recorre $N(v)$; el **sorted** local de la vecindad cuesta $\deg(v) \log \deg(v)$, y sumado da

$$T(n, m) = \Theta(n) + \sum_{v \in V} \mathcal{O}(\deg(v) \log \deg(v)) + \Theta(n) \leq \Theta(n) + \log \Delta \sum_v \deg(v) = \mathcal{O}(n + m \log \Delta).$$

Sin el desempate por grado (un BFS puro) sería $\Theta(n + m)$; el **sorted** es una mejora de calidad opcional que a lo sumo añade el factor $\log \Delta$. La versión **bfs_levels** (que devuelve la distancia en aristas de s a cada vértice, usada por el mapa para el sombreado verde→rojo) es un BFS estándar en $\Theta(n + m)$:

```

1 def bfs_levels(g, source):
2     dist = {v: -1 for v in g.vertices()}; dist[source] = 0
3     queue = deque([source])
4     while queue: # cada vertice 1 vez
5         v = queue.popleft()
6         for u in g.neighbors(v): # sum deg(v) = 2m
7             if dist[u] == -1:
8                 dist[u] = dist[v] + 1; queue.append(u)
9     return dist

```

Colorear golosamente en **order_distance** produce un coloreo propio (mismo argumento que en §4) cuyo frente de color se expande por oleadas desde s : es la animación “por distancia” de la interfaz.

4. greedy

```

1 def greedy(graph, order):
2     color = {v: 0 for v in graph.vertices()} # B1: n
3     for v in order: # B2: n iter.
4         used = {color[u] for u in graph.neighbors(v) if color[u]} # B3: deg(v)
5         c = 1 # B4
6         while c in used: c += 1 # B5: <= deg(v)+1
7         color[v] = c # B6
8     return color

```

Cota del while: $|\text{used}| \leq \deg(v)$, luego entre $1, \dots, \deg(v)+1$ hay un color libre (casillero); B_5 itera $\leq \deg(v) + 1$ veces con membresía $\mathcal{O}(1)$. Sumando con (1):

$$T(n, m) = T_{\text{orden}} + c_1 n + \sum_{v \in V} [c_3 \deg(v) + c_4 + c_5(\deg(v)+2) + c_6] = T_{\text{orden}} + \Theta(n + m).$$

Con cualquier orden $T_{\text{orden}} = \mathcal{O}(n + m \log \Delta)$, luego

$$T_{\text{greedy}} = \Theta(n + m) \text{ (órdenes natural/LF/SL), } \mathcal{O}(n + m \log \Delta) \text{ (distancia)}$$

espacio extra $\mathcal{O}(n) + \mathcal{O}(\Delta)$.

Teorema 4.1. *Greedy produce un coloreo propio con $\leq \Delta(G) + 1$ colores.*

Observación 4.2. El *orden* importa: en el Perú, el orden natural usa 5 colores mientras que mayor-grado, smallest-last, distancia y DSATUR alcanzan $4 = \chi(G)$ (§10).

5. dsatur

DSATUR colorea siempre el vértice no coloreado de mayor *saturación* $\text{sat}(v) = |\{\text{colores en } N(v)\}|$, desempataando por grado. Se usa un heap binario con **borrado perezoso**: nunca se elimina del heap; al extraer se descartan entradas de vértices ya coloreados o con saturación vieja.

```

1 def dsatur(graph):
2     color = {v: 0 for v in graph.vertices()}           # B1: n
3     sat = {v: set() for v in graph.vertices()}         # B1: n
4     heap = [(0, -graph.degree(v), v) for v in graph.vertices()]
5     heapq.heapify(heap); pending = graph.n             # B1: heapify Theta(n)
6     while pending:                                     # B2
7         neg_sat, _, v = heapq.heappop(heap)            # B2: O(log n)
8         if color[v] or -neg_sat != len(sat[v]): continue # B2: entrada obsoleta
9         c = 1
10        while c in sat[v]: c += 1                      # B3: <= sat(v)+1
11        color[v] = c; pending -= 1
12        for u in graph.neighbors(v):                   # B4: deg(v)
13            if not color[u] and c not in sat[u]:
14                sat[u].add(c)
15                heapq.heappush(heap, (-len(sat[u]),
16                                     -graph.degree(u), u)) # B4: O(log n)

```

Lema 5.1 (entrada fresca). *Para cada vértice no coloreado hay siempre en el heap una entrada con su saturación actual, y ninguna con saturación mayor.*

Demostración. Inducción: la entrada inicial $(0, \cdot, v)$ es exacta; cada aumento de $\text{sat}(v)$ inserta de inmediato una entrada nueva (las viejas quedan menores). La fresca solo desaparece al extraerse, y entonces v se colorea. $\square$

Como el heap extrae el mínimo de $(-\text{sat}, -\text{deg}, v)$ (máxima saturación) y la entrada fresca tiene la clave menor, el vértice procesado siempre es un no coloreado de saturación máxima: la regla de Brélaz. Nótese que $\text{sat}[v]$ es el conjunto de colores prohibidos, así que B_3 halla el menor color libre sin recorrer otra vez la vecindad.

Conteo agregado. Inserciones: n iniciales más a lo más una por cada par (arista, extremo), pues al colorear v el bloque B_4 revisa $\text{deg}(v)$ vecinos: $I \leq n + \sum_v \text{deg}(v) = n + 2m$. Extracciones $E \leq I$. B_3 suma $\sum_v (\text{sat}(v)+1) \leq n + 2m$; B_4 suma $2m$. Cada operación de heap sobre $\leq n + 2m$ entradas cuesta $\mathcal{O}(\log n)$ (pues $\log(n + 2m) \leq 2 \log n + 1$). Total

$$T(n, m) = \Theta(n) + (I + E) \mathcal{O}(\log n) + \Theta(n + m) = \boxed{\mathcal{O}((n + m) \log n)}, \quad \text{espacio } \Theta(n + m).$$

Proposición 5.2 (Brélaz). *DSATUR es exacto en grafos bipartitos (2 colores si $m \geq 1$); heurística en general.*

6. backtracking: chi(G) exacto

Teorema 6.1 (Karp 1972). *Decidir $\chi(G) \leq k$ es NP-completo para $k \geq 3$. No se conoce algoritmo polinomial exacto.*

6.1. greedy_clique: cota inferior polinomial

Una clique Q obliga a $|Q|$ colores: $\chi(G) \geq |Q|$. Se construye golosamente desde varias semillas de alto grado; con $s = \min(n, 100)$ semillas y $\omega_Q = |Q|$:

$$T = \mathcal{O}(n \log n) + \sum_{\text{seed}} [\mathcal{O}(\Delta \log \Delta) + \sum_{u \in N(\text{seed})} \mathcal{O}(\omega_Q)] = \mathcal{O}(n \log n + s \Delta (\log \Delta + \omega_Q)),$$

polinomial y despreciable frente a la búsqueda.

6.2. Podas (preservan completitud)

Lema 6.2 (P1 pre-coloreo de la clique). *Si existe una k -coloración, existe una con $q_i \mapsto i$ sobre la clique $Q = \{q_1, \dots, q_r\}$. (Los q_i reciben colores distintos; se permutan los nombres.)*

Lema 6.3 (P2 cota canónica). *Restringir el color de v_i a $\{1, \dots, 1 + \max_{j < i} \varphi(v_j)\}$ no pierde soluciones. (Renombrar colores por orden de primera aparición.)*

Lema 6.4 (P3 forward checking). *Quitar el color c del dominio de los vecinos no coloreados de v al asignar $\varphi(v)=c$, y podar si un dominio queda vacío, no pierde soluciones.*

6.3. `_search`: recurrencia del árbol

Los dominios se representan como **máscaras de bits** (un entero por vértice), de modo que quitar/reponer un color es $\mathcal{O}(1)$.

```

1 def extend(i, max_used):                                # un nodo del arbol
2     nodes[0] += 1                                       # B0:  $\mathcal{O}(1)$  + reloj
3     if i == n: return True
4     v = order[i]
5     allowed = dom[v] & mask(1 .. min(max_used+1, k))   # P2 + P3
6     while allowed:                                     # B1:  $\leq \min(k, i-q+1)$  colores
7         c = menor bit de allowed
8         color[v] = c
9         touched = propagate(v, c)                      # B2:  $\mathcal{O}(\deg(v))$  forward checking
10        if touched is not None:
11            if extend(i+1, max(max_used, c)): return True
12            reponer touched                             # B2:  $\mathcal{O}(\deg(v))$ 
13            color[v] = 0                                # backtrack
14    return False

```

`propagate` recorre $N(v)$ una vez ($\Theta(\deg(v))$). Sea b_i el factor de ramificación en la posición i ($b_i \leq \min(k, q+i+1)$ por P1+P2). El número de nodos a profundidad t es $\leq \prod_{j \leq t} b_j \leq k^t$, y el total

$$N_{\text{nodos}} \leq \sum_{t=0}^{n-q} k^t = \frac{k^{n-q+1} - 1}{k - 1} = \mathcal{O}(k^{n-q}).$$

Con $\mathcal{O}(\Delta + k)$ de trabajo por nodo,

$$T_{\text{search}} = \mathcal{O}((\Delta + k) k^{n-q}) \subseteq \mathcal{O}(m k^n)$$

— exponencial, como corresponde a NP-completo. Espacio $\Theta(n + m)$: dominios, color y pila de recursión con listas de deshacer que suman $\leq 2m$ por rama.

6.4. `chromatic_number`: iterar sobre k

Se arranca en $lb = |\text{clique}|$ y se sube hasta $ub = \text{colores}(\text{DSATUR})$; si $lb = ub$ no hay búsqueda. El primer k con éxito es $\chi(G)$ (todas las $k < \chi$ fallaron o $\chi = lb$). Costo

$$T = \underbrace{\mathcal{O}((n+m) \log n)}_{\text{DSATUR}} + \underbrace{\mathcal{O}(n \log n + \Delta^2)}_{\text{clique}} + \sum_{k=lb}^{\chi} \mathcal{O}((\Delta+k) k^{n-q}) = \mathcal{O}((\Delta+\chi) \chi^{n-q}),$$

dominado por $k = \chi$. **Mejor caso** ($lb = ub$, como en el Perú): polinomial, 0 nodos. Las guardas `max_n` y `timeout_s` convierten el peor caso en un fallo controlado que reporta $[lb, ub]$ y la mejor coloración conocida.

Efecto de las podas (medido). En el mapa del Perú, decidir “ $\exists$ 4-coloración” cierra con $lb = ub = 4$ y **0 nodos**; el backtracking ciego exploraría $\sim 4^{21}$. En $G(1000, p)$ la búsqueda agota su presupuesto: la explosión exponencial es real (§10).

7. generators.py: instancias aleatorias

7.1. gnp: $G(n, p)$ en $\Theta(n + m)$ esperado

El método ingenuo sortea los $\binom{n}{2}$ pares ($\Theta(n^2)$, inviable para $n = 10^6$). El muestreo de saltos (Batagelj–Brandes 2005) genera directamente la posición de la siguiente arista:

```

1 log_q = math.log1p(-p) # ln(1-p)
2 v, w = 1, -1
3 while v < n: # una iter. por arista (+ n)
4     w += 1 + int(math.log1p(-rng.random()) / log_q) # salto ~ Geo(p)
5     while w >= v and v < n: w -= v; v += 1 # normalizar al renglon
6     if v < n: g.add_edge(v, w) # O(1)

```

Salto geométrico: recorriendo los pares en orden lexicográfico, la distancia S hasta el siguiente presente cumple $\Pr[S=s] = (1-p)^s p$; por inversión de la CDF, $S = \lfloor \ln(1-r)/\ln(1-p) \rfloor$ con $r \sim U[0, 1]$ — una muestra $\mathcal{O}(1)$ por arista. El bucle interno avanza v en total n veces (amortizado). Así $\mathbb{E}[T] = \Theta(n + \mathbb{E}[m])$, con $\mathbb{E}[m] = \binom{n}{2}p$; con $p = d/(n-1)$, $\mathbb{E}[m] = nd/2 = \Theta(n)$.

7.2. geometric: rejilla espacial

n puntos uniformes en $[0, 1]^2$, arista si la distancia es $\leq r$. Una rejilla de celdas $r \times r$ restringe las comparaciones a celdas vecinas (estencil de 5 desplazamientos, cada par de celdas una vez). El número de puntos por celda es binomial de media $\lambda = nr^2$; con $r = \sqrt{d/(\pi n)}$ (grado medio d), $\lambda = d/\pi = \mathcal{O}(1)$, y los pares examinados son

$$\mathbb{E}\left[\sum_{\text{celdas}} \left(\binom{|c|}{2} + 4|c||c'|\right)\right] = \frac{1}{r^2} \mathcal{O}(\lambda^2) = \mathcal{O}(n\lambda) = \mathcal{O}(nd) = \Theta(n).$$

Costo esperado $\Theta(n + m)$: el modelo FAP natural (antenas cercanas interfieren).

8. peru.py: el mapa

`build_peru_graph` carga `data/peru_map.json` (polígonos, centroides y lista de fronteras derivados de un GeoJSON oficial) y llama a `from_edges`: $\Theta(n + m)$, que aquí es constante ($n=25$, $m=53$). La adyacencia se obtuvo geoméricamente: dos departamentos son vecinos si sus polígonos comparten ≥ 2 vértices de frontera; el resultado coincide con la verificación manual (Lima, Cusco y Huánuco con grado 7; Callao y Tumbes con grado 1). `geo_distance` aplica la fórmula de Haversine sobre los centroides en $\mathcal{O}(1)$.

Ruteo del grafo por carreteras (visualización). Para dibujar las aristas “siguiendo las carreteras del Perú” se descargó la red vial nacional de OpenStreetMap (vías *motorway/trunk/primary*, $\sim 1.3\text{M}$ de puntos), se construyó el grafo vial (nodo por coordenada, arista por tramo con peso de Haversine) y se ubicó cada capital departamental. Con un **Dijkstra multi-fuente** desde las 25 capitales (`scipy.sparse.csgraph.dijkstra`, $\mathcal{O}(E \log V)$ por fuente) se obtuvo, para cada arista de interferencia (u, v) , el camino mínimo por carretera entre las capitales de u y v , que se guarda como polilínea en `data/peru_map.json`. **49 de las 53 aristas** siguen carreteras reales; las 4 restantes son las fronteras de Loreto (Iquitos no tiene conexión vial con el resto del país) y se dibujan como segmento recto. Esto es puramente *visual*: el modelo de coloreo sigue siendo la adyacencia por fronteras (§8.1), y por tanto $\chi = 4$ no cambia.

Teorema 8.1. $\chi(\text{Perú}) = 4$.

Demostración. ≥ 4 : el grafo contiene el K_4 {Cusco, Apurímac, Ayacucho, Arequipa}. ≤ 4 : existe la 4-coloración hallada por el backtracking (certificado verificable con `is_proper` en $\Theta(n + m)$). Todo mapa plano admite 4 colores (Appel–Haken 1977), pero el par (clique, coloración) es una demostración autocontenida para *este* grafo. $\square$

9. Tabla resumen

Función	Tiempo	Espacio extra	Técnica
add_vertex/add_edge	$\mathcal{O}(1)$	—	costo directo (hash)
edges	$\Theta(n + m)$	$\mathcal{O}(1)$	sumatoria + (1)
is_proper	$\Theta(n + m)$	$\mathcal{O}(1)$	sumatoria sobre aristas
order_natural	$\Theta(n)$	$\Theta(n)$	copia
order_largest_first	$\Theta(n + \Delta)$	$\Theta(n)$	bucket sort
order_smallest_last	$\Theta(n + m)$	$\Theta(n)$	amortizado (lema 3.1)
order_distance	$\mathcal{O}(n + m \log \Delta)$	$\Theta(n)$	BFS + orden local
bfs_levels	$\Theta(n + m)$	$\Theta(n)$	BFS
greedy	$\Theta(n + m)$	$\Theta(n)$	sumatoria + casillero
dsatur	$\mathcal{O}((n+m) \log n)$	$\Theta(n + m)$	agregado + borrado perezoso
greedy_clique	$\mathcal{O}(n \log n + \Delta^2)$	$\Theta(n)$	sumatoria por semilla
_search	$\mathcal{O}((\Delta+k)k^{n-q})$	$\Theta(n + m)$	árbol de recursión
chromatic_number	$\mathcal{O}((\Delta+\chi)\chi^{n-q})$	$\Theta(n + m)$	serie geométrica
gnp	$\Theta(n + m)$ esp.	$\Theta(n + m)$	esperanza + inversión CDF
geometric	$\Theta(n + m)$ esp.	$\Theta(n + m)$	esperanza (rejilla)
build_peru_graph	$\Theta(n + m)$	$\Theta(n + m)$	carga de datos

10. Contraste experimental

La aplicación (`python main.py`) reúne el mapa interactivo, el benchmark, el uso de recursos, la suite de pruebas y la demo. La Fig. 1 muestra las dos vistas del mapa: coloreo por frecuencias (propio, $\chi = 4$) y sombreado por distancia desde un departamento.

Mapa del Perú coloreado por frecuencias (DSATUR, $\chi=4$)



Coloreo por distancia desde Lima (verde=cerca, rojo=lejos)



Figura 1: Izquierda: coloreo por frecuencias (DSATUR, $\chi = 4$). Derecha: coloreo por distancia (BFS) desde Lima; verde=cerca, rojo=lejos.

En la gráfica log-log (Fig. 3) una función $\Theta(n^a)$ es una recta de pendiente a : Greedy y DSATUR sobre grafos geométricos dispersos ($m = \Theta(n)$) siguen la pendiente 1 de la referencia $\Theta(n)$.

Observaciones: (i) en el Perú todos los órdenes salvo el natural alcanzan $\chi = 4$; (ii) DSATUR usa menos colores que los greedy en $G(n, p)$ a costa de más tiempo; (iii) el backtracking es exacto e instantáneo en $n = 25$ (0 nodos: las cotas cierran) pero agota su presupuesto en $n = 10^3$ —la frontera práctica de la exactitud; (iv) solo Greedy y DSATUR escalan a $n = 10^6$, coherente con sus cotas casi lineales. En los n grandes, smallest-last y DSATUR se desvían de la pendiente 1: no porque su conteo de operaciones deje de ser lineal (las sumatorias de §3.3 y §5 lo garantizan), sino porque el costo por operación crece (cubetas de conjuntos y heap dispersan los accesos a memoria, más el factor $\log n$ de DSATUR): el modelo RAM de costo uniforme no captura la caché.

Grafo de interferencia ruteado por carreteras reales (OSM)
rojo = carretera nacional · azul discontinuo = sin conexión vial



Figura 2: Grafo de interferencia con las aristas ruteadas sobre la red vial real (OSM): rojo = carretera nacional; azul discontinuo = par sin conexión vial (fronteras de Loreto). Los nodos se ubican en las capitales departamentales.

11. El caso $n = 10^{10}$: por qué es solo teórico

- **Memoria:** el arreglo de colores a 2 bytes (`uint16`) ocupa $2 \cdot 10^{10}$ B = 20 GB; la adyacencia con grado medio 6 a 8 bytes, ~ 480 GB. El modelo RAM deja de aplicar (no cabe en memoria).
- **Tiempo extrapolado:** la curva medida hasta 10^6 tiene pendiente ≈ 1 ; escalar $\times 10^4$ daría horas *si* la memoria fuera plana, pero con paginación a disco domina la E/S.
- **Conclusión:** 10^{10} es una cota asintótica, no un experimento; se aborda con computación distribuida (BSP/Pregel, grafo particionado) o memoria externa. El backtracking queda descartado por tiempo exponencial.

Referencias

- [1] Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4.^a ed., MIT Press, 2022.
- [2] D. Brélaz. *New methods to color the vertices of a graph*. CACM 22(4), 1979.
- [3] D. Welsh, M. Powell. *An upper bound for the chromatic number of a graph*. Computer Journal 10(1), 1967.
- [4] D. Matula, L. Beck. *Smallest-last ordering and clustering and graph coloring algorithms*. JACM 30(3), 1983.
- [5] R. Karp. *Reducibility among combinatorial problems*, 1972.
- [6] V. Batagelj, U. Brandes. *Efficient generation of large random networks*. Phys. Rev. E 71, 036113, 2005.
- [7] K. Appel, W. Haken. *Every planar map is four colorable*. Illinois J. Math. 21, 1977.

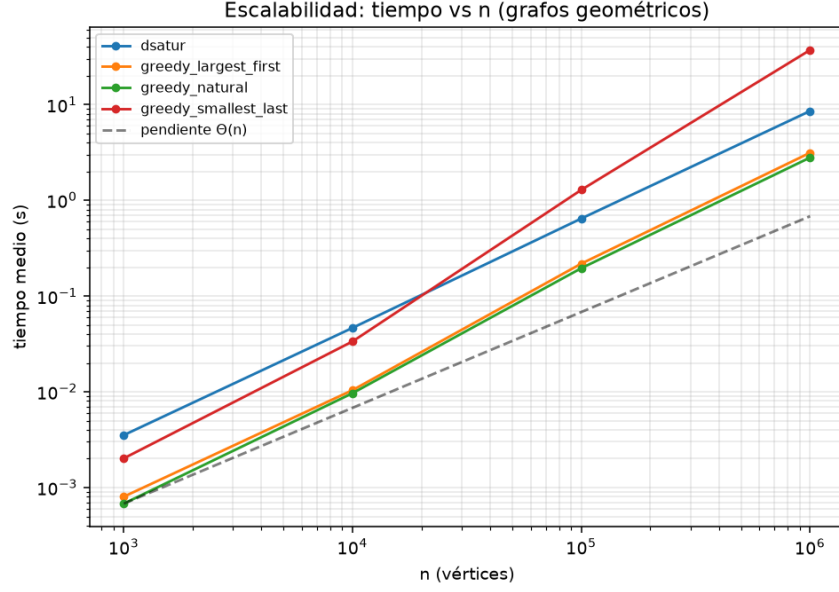


Figura 3: Tiempo medio vs n (log-log). La recta discontinua es $\Theta(n)$.

Tabla 1: Resultados experimentales (media de repeticiones, semilla 42; backtracking con timeout de 30 s).

Instancia	Algoritmo	n	m	Colores	Tiempo	Mem.
Peru	Greedy natural	25	53	5	26 μ s	0.00 MB
Peru	Greedy mayor grado	25	53	4	44 μ s	0.00 MB
Peru	Greedy smallest-last	25	53	4	64 μ s	0.01 MB
Peru	DSATUR	25	53	4	64 μ s	0.01 MB
Peru	Backtracking	25	53	4 (chi=4 exacto)	137 μ s	—
gnp_1000	Greedy natural	1000	4995	8	957 μ s	0.06 MB
gnp_1000	Greedy mayor grado	1000	4995	7	1.38 ms	0.06 MB
gnp_1000	Greedy smallest-last	1000	4995	7	2.99 ms	0.18 MB
gnp_1000	DSATUR	1000	4995	5	4.16 ms	0.64 MB
gnp_1000	Backtracking	1000	4995	5 (chi en [3])	30.00 s	—
geo_1000	Greedy natural	1000	2896	9	680 μ s	0.06 MB
geo_1000	Greedy mayor grado	1000	2896	9	803 μ s	0.06 MB
geo_1000	Greedy smallest-last	1000	2896	9	2.02 ms	0.14 MB
geo_1000	DSATUR	1000	2896	9	3.53 ms	0.44 MB
geo_10000	Greedy natural	10000	30043	11	9.67 ms	0.50 MB
geo_10000	Greedy mayor grado	10000	30043	10	10.40 ms	0.51 MB
geo_10000	Greedy smallest-last	10000	30043	10	33.52 ms	1.62 MB
geo_10000	DSATUR	10000	30043	10	46.46 ms	4.28 MB
geo_100000	Greedy natural	100000	298939	12	194.93 ms	8.26 MB
geo_100000	Greedy mayor grado	100000	298939	11	217.43 ms	8.27 MB
geo_100000	Greedy smallest-last	100000	298939	11	1.28 s	14.85 MB
geo_100000	DSATUR	100000	298939	11	643.15 ms	45.96 MB
geo_1000000	Greedy natural	1000000	2996912	13	2.78 s	67.63 MB
geo_1000000	Greedy mayor grado	1000000	2996912	13	3.12 s	68.06 MB
geo_1000000	Greedy smallest-last	1000000	2996912	13	36.92 s	124.23 MB
geo_1000000	DSATUR	1000000	2996912	13	8.53 s	442.39 MB